Group members:

Student Names and IDs:

Maimuna Aminu Suleiman 210208950

SIDI LAMINE ABDOURAHMANE 220208889

Zakariyya Zakariyya Suleiman 230208857

SecureShield: Role-Based Access Control API

Introduction

In modern web applications, authentication and authorization are essential for protecting user accounts and restricting access to sensitive operations. This project presents the implementation of a Python-based Flask API called *SecureShield*. The system uses bcrypt for secure password storage, JSON Web Tokens (JWT) for authentication, and Role-Based Access Control (RBAC) to separate normal user permissions from administrator permissions.

The project is divided into two main parts. Part 1 focuses on authentication and identity, while Part 2 focuses on access control and authorization.

Part 1: Authentication and Identity

The first part of the project focuses on verifying user identity securely. The system provides user registration, login, and protected route access using JWT.

During registration, user passwords are not stored in plain text. Instead, passwords are hashed using bcrypt before being saved in the SQLite database. This protects users even if the database is exposed.

During login, the system checks the submitted password against the stored bcrypt hash. If the credentials are correct, the server generates a JWT containing the user ID, username, role, and expiration time. This token is then used to access protected routes.

The system performs the following operations:

- Registers users securely
- Hashes and salts passwords using bcrypt
- Authenticates users during login
- Issues JWT tokens after successful login
- Validates tokens before allowing access to protected routes

Why Salting is Necessary to Prevent Rainbow Table Attacks

Hashing alone is not enough to protect passwords because attackers can use rainbow tables. A rainbow table is a precomputed list of common passwords and their hash values. If a database contains unsalted password hashes, attackers can compare the stolen hashes with the rainbow table and quickly discover the original passwords.

Salting solves this problem by adding a unique random value to each password before hashing. This means that even if two users have the same password, their stored hashes will be different.

Salting is important because:

- It prevents identical passwords from producing identical hashes
- It makes precomputed rainbow tables ineffective
- It forces attackers to attack each password hash individually
- It increases the time and cost required to crack passwords

In this project, bcrypt is used because it automatically handles salting and hashing securely.

Part 2: Access Control and Authorization

The second part of the project focuses on controlling what authenticated users are allowed to do. The system uses Role-Based Access Control (RBAC), where users are assigned either the `User` role or the `Admin` role.

The `/profile` endpoint is accessible to both normal users and administrators. However, the `DELETE /user/<id>` endpoint is restricted to administrators only. If a normal user attempts to access this admin-only route, the system returns a `403 Forbidden` response.

The system also includes logout functionality using token blacklisting. Since JWTs are stateless, they normally remain valid until they expire. To solve this, logged-out tokens are stored in a blacklist. If a blacklisted token is used again, the server rejects it.

The system performs the following operations:

- Allows both User and Admin roles to access `/profile`
- Restricts `DELETE /user/<id>` to Admin users only
- Returns `403 Forbidden` for unauthorized role access
- Logs unauthorized admin-route attempts in `security.log`
- Invalidates logged-out tokens using a blacklist

Risks of Storing Sensitive Data in JWT

JWTs are useful for authentication, but sensitive information should not be stored inside them. A JWT payload is Base64 encoded, not encrypted. This means anyone who has the token can decode it and view the payload.

Storing sensitive data such as passwords, national IDs, private keys, or personal information inside a JWT is dangerous because:

- The token may be stored on the client side
- If the token is stolen, the attacker can read the payload
- Sensitive data can leak even if the token signature is valid
- A stolen token can be reused until it expires if proper controls are not in place

For this reason, this project only stores minimal information in the JWT, such as user ID, username, role, and expiration time. Passwords are never stored inside the token.

System Workflow

Registration

1. User submits username, password, and role.
2. The password is hashed and salted using bcrypt.
3. The user record is stored in the SQLite database.

Login

1. User submits username and password.
2. The server verifies the password against the stored hash.
3. If valid, the server generates a signed JWT.

Protected Access

1. The client sends the JWT in the Authorization header.
2. The server validates the token signature and expiration.
3. If valid, access is granted based on the user's role.

Authorization

1. User and Admin can access `/profile`.
2. Only Admin can access `DELETE /user/<id>`.
3. Unauthorized attempts are logged in `security.log`.

Logout

1. User sends a logout request with their token.
2. The token is added to a blacklist.
3. Any future request using that token is rejected.

Conclusion

The SecureShield project demonstrates how authentication and authorization can be implemented in a secure backend system. Bcrypt protects passwords through hashing and salting, reducing the risk of rainbow table attacks. JWT provides a practical way to authenticate users, while RBAC ensures that users only access features permitted by their role.

The project also improves security by using token blacklisting for logout and defensive logging for unauthorized access attempts. Overall, SecureShield follows the principle of least privilege by ensuring that standard users cannot access administrator-only functionality.